

# Programiranje za web

## Predavanje 0: O predmetu

doc. dr. sc. Tomislav Petković   mag. ing. Josip Torić

08.10.2026.



**Sveučilište u Zadru**  
Universitas Studiorum  
Jadertina | 1396 | 2002 |

# Plan predavanja

- 1 Predmet kolegija i ishodi učenja
- 2 Referentna aplikacija: prijave klubova na natjecanje
- 3 Arhitektura i tehnologije
- 4 Organizacija nastave i vrednovanje
- 5 Radna okolina i način rada

Kolegij je organiziran oko jedne aplikacije koja se razvija od prve do posljednje nastavne jedinice: podatkovni model, poslovna pravila, korisničko sučelje i postavljanje u produkciju.

---

# Kolegij

Predmet kolegija, ishodi učenja, pretpostavljeno znanje.

# Predmet kolegija

Web aplikacija je program kojemu korisnik pristupa preglednikom, a čiji podatci ostaju pohranjeni i nakon što korisnik zatvori karticu. Iz toga slijede tri odvojena dijela:

- **frontend** (klijentski dio): prikaz podataka u pregledniku i unos korisnika
- **backend** (poslužiteljski dio): provjera identiteta i prava, poslovna pravila, pristup podacima
- **baza podataka**: trajna pohrana i ograničenja nad podacima

Kolegij obrađuje sva tri dijela i sučelje kojim međusobno komuniciraju.

# Ishodi učenja

Nakon položenog kolegija student može:

- modelirati domenu i preslikati je u relacijsku bazu zajedno s ograničenjima
- oblikovati programsko sučelje API (engl. *application programming interface*) u stilu REST (engl. *representational state transfer*) i validirati ulazne podatke
- implementirati autentikaciju tokenom JWT (engl. *JSON Web Token*) te autorizaciju po ulozi i po vlasništvu zapisa
- izraditi jednostraničnu aplikaciju (engl. *single-page application*, SPA) koja se oslanja na taj API
- napisati testove poslovnih pravila i postaviti aplikaciju u produkcijsku okolinu

# Pretpostavljeno znanje

- programiranje u jeziku opće namjene; sintaksa Pythona uvodi se usput
- osnove relacijskih baza: tablica, primarni i strani ključ, upit SELECT
- osnove HTML-a i CSS-a
- rad s Gitom: `commit`, `branch`, `remote`

Nedostatak jednog od navedenih znanja nije prepreka, ali traži više samostalnog rada u prvim tjednima.

---

# Referentna aplikacija

Problem prijava na natjecanje, model domene, faze i rokovi, svojstva domene relevantna za kolegij.

## Problem: prijave klubova na natjecanje

Organizator powerlifting natjecanja prikuplja prijave od klubova. Bez aplikacije prijave stižu e-poštom i tablicama u Excelu, pa organizator ručno provjerava:

- je li prijava stigla prije isteka roka
- prijavljuje li klub natjecatelja koji mu pripada
- je li isti natjecatelj prijavljen dvaput
- odgovara li težinska kategorija spolu i tjelesnoj masi natjecatelja

Svaka od tih provjera je pravilo koje aplikacija treba provoditi umjesto čovjeka. Ista se aplikacija razvija kroz cijeli kolegij i služi kao predložak za studentski projekt.

# Model domene

## Entiteti

- **Klub:** prijavljuje natjecatelje
- **Korisnik:** račun za prijavu, uloga admin ili club; svaki klub ima jedan
- **Natjecatelj:** pripada točno jednom klubu
- **Natjecanje:** datum održavanja i dva roka
- **Prijava:** veza natjecatelja i natjecanja

Kategorije IPF, muškarci: 59, 66, 74, 83, 93, 105, 120 i 120+ kg; žene: 47, 52, 57, 63, 69, 76, 84 i 84+ kg.

## Pravila nad modelom

- klub vidi i mijenja samo svoje natjecatelje i svoje prijave
- organizator vidi prijave svih klubova
- isti natjecatelj ne može imati dvije aktivne prijave na istom natjecanju
- težinska kategorija bira se iz popisa kategorija saveza IPF

## Faze natjecanja i rokovi

Natjecanje ima dva roka nominacije: **preliminarni** (`prelim_deadline`) i **finalni** (`final_deadline`). Oni dijele vrijeme na tri faze s različitim skupom dopuštenih operacija.

| Faza          | Vrijeme               | Nova prijava | Promjena kat. | Odjava |
|---------------|-----------------------|--------------|---------------|--------|
| OPEN          | do preliminarnog roka | da           | da            | da     |
| PRELIM_PASSED | između dvaju rokova   | ne           | da            | da     |
| CLOSED        | nakon finalnog roka   | ne           | ne            | ne     |

Preliminarni rok organizatoru daje popis potreban za raspored vaganja i natjecateljskih skupina; nakon finalnog roka popis se više ne mijenja.

## Radni primjer: četiri zahtjeva istoga kluba

Natjecanje se održava 14.03.2027.; preliminarni rok istječe 14.02.2027. u 23:59 (28 dana ranije), finalni 07.03.2027. u 23:59 (7 dana ranije).  
Ilustrativno.

| Trenutak zahtjeva | Zahtjev                          | Faza          | Odgovor      |
|-------------------|----------------------------------|---------------|--------------|
| 10.02.2027.       | prijava natjecatelja, kat. 83 kg | OPEN          | 201 Created  |
| 20.02.2027.       | promjena kategorije 83 → 93 kg   | PRELIM_PASSED | 200 OK       |
| 20.02.2027.       | prijava novog natjecatelja       | PRELIM_PASSED | 409 Conflict |
| 09.03.2027.       | odjava natjecatelja              | CLOSED        | 409 Conflict |

Statusni kod 409 Conflict znači da su poslani podatci ispravni, ali je operacija u sukobu sa stanjem resursa. Ishod ovisi o trenutku zahtjeva, a ne o tome što sučelje prikazuje.

# Svojstva domene relevantna za kolegij

- dvije uloge koje vide različit skup podataka, pa se autorizacija provjerava pri svakom zahtjevu
- vlasništvo nad zapisom: klub A ne smije dohvatiti natjecatelja kluba B
- pravila ovisna o vremenu: isti zahtjev prolazi ili ne prolazi ovisno o fazi
- relacije i ograničenja jedinstvenosti, koja provodi baza, a ne aplikacijski kod
- sučelje s tablicama, filtrima i dostupnošću naredbi ovisnom o fazi

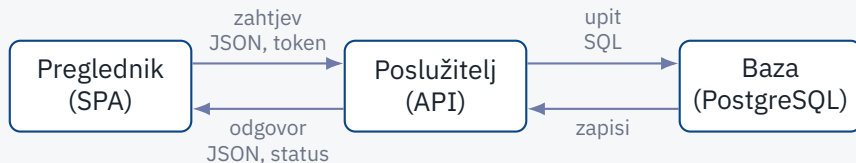
CRUD (engl. *create, read, update, delete*) označava četiri osnovne operacije nad zapisom. Aplikacija koja izvodi samo njih ne pokriva nijedno od navedenih svojstava.

---

# Arhitektura i tehnologije

Troslojna arhitektura, podjela odgovornosti, tehnologije, struktura repozitorija.

# Troslojna arhitektura



Preglednik ne pristupa bazi izravno. Svaki podatak koji korisnik vidi prošao je kroz backend, koji je pritom provjerio tko je korisnik i što smije.

# Podjela odgovornosti

- Baza je izvor istine o podacima; ograničenja zapisana u shemi provode se neovisno o tome koji ih kod pokušava prekršiti.
- Backend je mjesto na kojem se pravilo provodi: prije svake izmjene provjeravaju se identitet, uloga, vlasništvo nad zapisom i faza natjecanja.
- Frontend pravila prikazuje, ali ih ne provodi. Onemogućen gumb korisniku štedi uzaludan pokušaj, ali ne sprječava zahtjev poslan mimo sučelja.

Provjera ispravnosti te podjele: zahtjev poslan izravno na API, mimo korisničkog sučelja, mora dobiti isti odgovor kao zahtjev poslan iz preglednika. Ako ga ne dobije, pravilo je zapisano na krivome mjestu.

# Tehnologije

## Backend

FastAPI i Uvicorn (HTTP) | Pydantic (validacija) | SQLAlchemy 2.0 (ORM) | Alembic (migracije) | PostgreSQL

## Frontend

Vue 3, Vite i TypeScript | Vue Router (rute) | Pinia (stanje) | Axios (HTTP)

## Alati i okolina

Docker Compose (baza) | Git i GitHub Actions (CI) | pytest (testovi) | ruff (statička analiza) | Railway ili Render (postavljanje u rad)

Ovo je jedna moguća kombinacija tehnologija. Ista se načela primjenjuju i uz drukčiji izbor.

# Struktura repozitorija

## Backend

- `routers/`: rute i parametri zahtjeva
- `schemas/`: Pydantic modeli ulaza i izlaza
- `models/`: SQLAlchemy tablice
- `services/`: poslovna pravila
- `repositories/`: upiti nad bazom
- `core/`: konfiguracija, autentikacija, pogreške

## Frontend

- `src/views/`: stranice
- `src/components/`: komponente sučelja
- `src/stores/`: spremišta stanja (Pinia)
- `src/router/`: rute i njihove zaštite
- `src/api/`: omotač oko HTTP klijenta

Sloj se prepoznaje po tome što ga mijenja: promjena adrese dira `routers/`, promjena pravila `services/`, promjena oblika podataka `models/` i migracije.

---

# Organizacija nastave

Raspored predavanja, zahtjevi na studentski projekt, bodovanje, obrana, akademska čestitost.

# Raspored predavanja

| Pred. | Datum           | Tema                                               |
|-------|-----------------|----------------------------------------------------|
| 0     | 08.10.          | O predmetu                                         |
| 1     | 08.10.          | Arhitektura backenda i kostur projekta             |
| 2     | 15.10.          | Modeliranje podataka, ORM i migracije              |
| 3     | 22.10.          | Autentikacija i sigurnost (JWT)                    |
| 4     | 29.10.          | Autorizacija i vlasništvo nad zapisom              |
| 5     | 05.11.          | Oblikovanje API-ja i CRUD domene                   |
| 6     | 12.11.          | Poslovna pravila i faze u backendu                 |
|       | 19.11.          | <b>Obrana prvog dijela projekta</b>                |
| 7     | 26.11.          | Arhitektura jednostranične aplikacije (Vue)        |
| 8     | 03.12.          | Komunikacija s API-jem i tijek prijave korisnika   |
| 9     | 10.12.          | Upravljanje stanjem i stanja korisničkog sučelja   |
| 10    | 17.12.          | Tablice, filtri i administratorsko sučelje         |
|       | 24.12. i 31.12. | <i>Božićni i novogodišnji praznici</i>             |
| 11    | 07.01.          | Testiranje cjelovite aplikacije                    |
| 12    | 14.01.          | Produkcijska izgradnja i postavljanje u produkciju |
|       | 21.01.          | <b>Obrana drugog dijela projekta</b>               |

# Studentski projekt

Tijekom semestra svaki student izrađuje vlastitu aplikaciju, po uzoru na referentnu, ali u drugoj domeni. Zahtjevi:

- autentikacija tokenom i najmanje dvije uloge s različitim pravima
- najmanje dva entiteta povezana relacijom
- vlasništvo nad zapisom: korisnik vidi i mijenja samo svoje podatke
- najmanje jedno pravilo ovisno o stanju ili vremenu: status, faza ili rok
- sučelje sa stanjima učitavanja, pogreške i praznog skupa rezultata

Predaja je u dva dijela, svaki nosi 40 bodova: prvi obuhvaća backend, drugi cjelovitu aplikaciju postavljenu u rad. Studenti koji ne predaju tijekom nastave predaju u ispitnim rokovima u veljači.

# Primjeri tema projekta

- rezervacija termina: dvorane, treneri, konzultacije
- prijave na događanja: kapacitet, lista čekanja, evidencija dolaska
- praćenje zadataka: projekti, statusi, komentari
- planovi treninga: programi, izvedbe, napredak
- odobravanje dokumenata: zahtjev, razine odobrenja, verzije
- mali oglasnik: oglasi, poruke, spremljeni oglasi
- evidencija članstva: članovi, uplate, statusi

Tema mora sadržavati uloge i pravila. Aplikacija koja zapise samo unosi i prikazuje, bez pravila o tome tko što smije i kada, ne zadovoljava zahtjeve projekta.

# Bodovanje

| Komponenta                                                  | Bodovi     |
|-------------------------------------------------------------|------------|
| Prvi dio projekta (backend)                                 | 40         |
| Drugi dio projekta (cjelovita aplikacija postavljena u rad) | 40         |
| Usmeni ispit                                                | 20         |
| <b>Ukupno</b>                                               | <b>100</b> |

Prema pravilniku studija, za uspješno polaganje kolegija nužno je prisustvovati 80% predavanja.

# Obrana projekta

Obrana se sastoji od prikaza rada aplikacije, pregleda repozitorija i objašnjenja dijelova koda koje odabere nastavnik. Moguća je i manja izmjena uživo: dodavanje polja, migracija te prilagodba modela i sučelja.

## Primjer pitanja na obrani projekta

- Na kojem se mjestu provjerava vlasništvo nad zapisom?
- Kako backend odbija izmjenu nakon isteka roka?
- Zašto zahtjev vraća 401, a ne 403? Zašto 409, a ne 422?
- Gdje se na frontendu obrađuje odgovor 401?

Ocjenjuje se objašnjenje odluka ugrađenih u kod.

# Akademski čestitost i uporaba AI alata

Uporaba AI alata i vanjskih izvora dopuštena je. Uvjet je da student razumije i objasni svaki dio predanoga koda; na obrani se provjerava razumijevanje.

Za svaki preuzeti isječak koda treba znati:

- koji problem rješava i uz koje pretpostavke
- kako se ponaša u rubnim slučajevima: prazan ulaz, istekao token, istodobni zahtjevi
- koji ga test pokriva

Kod koji student ne može objasniti ne donosi bodove.

---

# Radna okolina

Potrebni alati, pokretanje projekta, kriteriji kvalitete koda, način rada.

# Potrebni alati

- Git
- Python 3.11 ili noviji
- Docker Desktop s alatom Docker Compose, za pokretanje baze
- Node.js 18 ili noviji
- uređivač koda po izboru; na predavanjima se koristi VS Code

Okolina se postavlja prije prvoga predavanja s kodom. Instalacija tijekom nastave oduzima vrijeme predviđeno za gradivo.

# Pokretanje projekta

```
# 1) Environment variables
cp .env.example .env # PowerShell: Copy-Item .env.example .env

# 2) Database (PostgreSQL in a container)
docker compose up -d db

# 3) Backend
cd api
python -m venv .venv
source .venv/bin/activate # PowerShell: .venv\Scripts\Activate.ps1
pip install -r requirements.txt
alembic upgrade head # apply migrations
python -m app.seed # seed data
uvicorn app.main:app --reload

# 4) Frontend
cd ../web
npm install
npm run dev
```

.env.example je u repozitoriju i služi kao popis varijabli; .env s vlastitim vrijednostima ostaje na računalu i ne ulazi u repozitorij. Nakon pokretanja backend odgovara na /health, a opis sučelja stoji na /docs.

# Kriteriji kvalitete koda

## Backend

- validacija svih ulaznih podataka
- jedinstven oblik odgovora s pogreškom
- pravila u sloju `services/`, a ne u ruti
- test za svako poslovno pravilo

## Frontend

- stanja učitavanja, pogreške i praznog skupa
- zaštita ruta prema ulozi korisnika
- validacija obrazaca prije slanja
- adresa API-ja iz varijable okoline

Objema stranama zajedničko je načelo: svako se pravilo zapisuje na jednome mjestu i odatle poziva, umjesto da se ponavlja u svakoj ruti ili komponenti.

# Način rada

- Nakon svakoga predavanja: ponoviti korake u vlastitom repozitoriju i usporediti razlike naredbom `git diff`.
- Kratak i čest rad daje bolji rezultat od dugoga rada neposredno prije roka predaje.
- Postupak pri pogrešci: pročitati poruku, izolirati najmanji primjer koji je izaziva, provjeriti zahtjev i odgovor (statusni kod i tijelo), pa tek onda mijenjati kod.
- Pitanje nastavnom osoblju postavlja se uz konkretan materijal: poruku pogreške te sadržaj zahtjeva i odgovora.

# Sažetak

- Web aplikacija ima tri dijela; pravila provodi backend, a baza je izvor istine o podacima.
- Referentna domena su prijave klubova na natjecanje: dvije uloge, vlasništvo nad zapisom te tri faze određene dvama rokovima.
- Faza određuje ishod zahtjeva: nova prijava prolazi samo u fazi OPEN, promjena kategorije i odjava i u fazi PRELIM\_PASSED, a nakon finalnoga roka nijedna izmjena.
- Studentski projekt nosi 90 od 100 bodova i predaje se u dva dijela, a dolasci na predavanja preostalih 10.
- Na obrani se ocjenjuje objašnjenje odluka ugrađenih u kod.

# Što slijedi

- **08.10.2026.** Predavanje 1, arhitektura backenda i kostur projekta: struktura repozitorija, baza u kontejneru, prva ruta API-ja.
- **15.10.2026.** Predavanje 2, modeliranje podataka, ORM i migracije.
- **19.11.2026.** Obrana prvoga dijela projekta.

Do sljedećega predavanja: postaviti radnu okolinu i otvoriti repozitorij projekta.